

# Sprawozdanie z projektu:

## Generacja obrazu wideo przez interfejs HDMI w układzie FPGA

Mateusz Brokos 275647

Szymon Blatkowski

18 czerwca 2026

### Spis treści

|          |                                                                          |           |
|----------|--------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Wstęp teoretyczny</b>                                                 | <b>2</b>  |
| 1.1      | Standard wyświetlania obrazu i parametry czasowe (VGA) . . . . .         | 2         |
| 1.2      | Standard transmisji cyfrowej TMDS i HDMI . . . . .                       | 2         |
| 1.3      | Środowisko sprzętowe: Płytki AUP-ZU3 i karta rozszerzeń . . . . .        | 3         |
| 1.4      | Architektura projektu i użyte moduły . . . . .                           | 3         |
| <b>2</b> | <b>Schemat blokowy projektu (Block Design)</b>                           | <b>3</b>  |
| <b>3</b> | <b>Weryfikacja symulacyjna (Testbench)</b>                               | <b>4</b>  |
| <b>4</b> | <b>Wdrożenie i weryfikacja sprzętowa</b>                                 | <b>4</b>  |
| 4.1      | Rozszerzenie projektu: Sprzętowy multiplexer obrazów testowych . . . . . | 5         |
| <b>5</b> | <b>Weryfikacja sprzętowa z użyciem analizatora ILA</b>                   | <b>8</b>  |
| <b>6</b> | <b>Podsumowanie i wnioski</b>                                            | <b>9</b>  |
| <b>7</b> | <b>Kody źródłowe</b>                                                     | <b>11</b> |
| 7.1      | Moduł główny (hdmi_top.vhd) . . . . .                                    | 11        |
| 7.2      | Plik środowiska testowego (tb_video.vhd) . . . . .                       | 13        |
| 7.3      | Plik obostrzeń projektowych (hdmi_pins.xdc) . . . . .                    | 14        |
| 7.4      | Moduł generatora czasów wideo (VideoTiming.vhd) . . . . .                | 15        |
| 7.5      | Moduł generatora obrazu (ImgGen.vhd) . . . . .                           | 17        |

# 1 Wstęp teoretyczny

## 1.1 Standard wyświetlania obrazu i parametry czasowe (VGA)

Podstawą wyświetlania obrazu na monitorach jest odpowiednie sterowanie procesem rysowania pikseli. Historycznie, w monitorach CRT, proces ten polegał na sterowaniu wiązką elektronów (plamką), która omiatała ekran linia po linii, od lewej do prawej i z góry na dół. Mimo ewolucji technologii i przejścia na ekrany cyfrowe (LCD, OLED), interfejsy wideo w dużej mierze zachowały ten sekwencyjny sposób transmisji danych.

Aby odbiornik (monitor) wiedział, kiedy przenieść rysowanie do nowej linii lub rozpocząć generowanie nowej klatki obrazu, wykorzystuje się sygnały synchronizacji poziomej (HSync) oraz pionowej (VSync). Czas pomiędzy aktywnym rysowaniem pikseli a impulsami synchronizacji wypełniają tzw. marginesy (Front Porch oraz Back Porch). Służą one jako bufor czasowy – dawniej pozwalały na powrót wiązki elektronów na początek ekranu, a dziś dają elektronice monitora czas na przetworzenie danych. W trakcie trwania marginesów i impulsów synchronizacji sygnał DE (Display Enable) znajduje się w stanie niskim, wygaszając generowanie kolorów.

W projekcie zrealizowano klasyczny tryb wideo o rozdzielczości 640x480 pikseli przy częstotliwości odświeżania 60 Hz. Częstotliwość zegara pikseli (Pixel Clock) nie wynika jednak bezpośrednio z pomnożenia tych trzech wartości, ponieważ należy doliczyć wspomniane strefy wygaszania (marginesy). Całkowity wymiar ramki wideo to 800 pikseli w poziomie (640 aktywnych + 16 Front Porch + 96 Sync Pulse + 48 Back Porch) oraz 521 linii w pionie (480 aktywnych + 10 Front Porch + 2 Sync Pulse + 29 Back Porch). Wynika z tego bezpośrednio częstotliwość odświeżania pikseli:

$$800\text{pikseli} \times 521\text{linii} \times 60\text{Hz} \approx 25\text{MHz}$$

Dlatego układ logiczny musi być taktowany rygorystycznym zegarem o częstotliwości 25 MHz.

## 1.2 Standard transmisji cyfrowej TMDS i HDMI

Zamiast przysyłać sygnały analogowe (jak w przypadku klasycznego kabla VGA), docelowym złączem w projekcie jest HDMI. Korzysta ono z technologii TMDS (Transition Minimized Differential Signaling). Przesyła ona cyfrowe dane o pikselach z bardzo dużą prędkością przy użyciu sygnałów różnicowych. Rozwiązanie to drastycznie minimalizuje zakłócenia elektromagnetyczne (EMI) i pozwala na transmisję na większe odległości.

Transmisja HDMI opiera się na trzech kanałach danych (odpowiadających w uproszczeniu kolorom RGB) oraz jednym kanale zegarowym. Sygnał równoległy musi zostać poddany serializacji. Ponadto, TMDS wykorzystuje kodowanie 8b/10b, co oznacza, że każdy 8-bitowy bajt koloru jest przekształcany w 10-bitowe słowo. Ma to na celu zbalansowanie składowej stałej (DC) na linii oraz minimalizację przejść logicznych, co ułatwia odzyskiwanie zegara po stronie odbiornika. Z tego powodu fizyczny nadajnik TMDS wymaga zegara taktującego o częstotliwości wielokrotnie wyższej niż podstawowy Pixel Clock (w tym przypadku 5-krotnie wyższego zegara z transmisją typu DDR - Double Data Rate, co daje 10-krotne przyspieszenie).

Standard TMDS optymalizuje wykorzystanie dostępnych kanałów transmisyjnych, dynamicznie zmieniając ich przeznaczenie w zależności od etapu rysowania klatki. W momencie, gdy sygnał DE (Display Enable) znajduje się w stanie niskim (trwają marginesy

wygaszania), nadajnik przechodzi w tryb kontrolny (Control Period). W tym oknie czasowym, zamiast informacji o pikselach, przesyłane są sygnały synchronizacji poziomej (HSync) i pionowej (VSync). Są one kodowane za pomocą unikalnych 10-bitowych tokenów kontrolnych i najczęściej multipleksowane w pierwszym kanale danych (Channel 0 / Blue). To inteligentne współdzielenie pasma całkowicie eliminuje konieczność prowadzenia osobnych, fizycznych linii sygnałowych dla synchronizacji w standardzie HDMI.

### 1.3 Środowisko sprzętowe: Płytki AUP-ZU3 i karta rozszerzeń

Prototypowanie układu zostało zrealizowane na zaawansowanej płycie ewaluacyjnej AUP-ZU3, której sercem jest układ FPGA z rodziny Zynq UltraScale+ (łączy programowalną matrycę bramek z procesorem ARM, choć w tym projekcie wykorzystano wyłącznie część logiczną – PL). Układ ten taktowany jest zewnętrznym zegarem głównym 100 MHz. Ze względu na brak wbudowanego złącza wideo na płycie bazowej, wykorzystano dedykowaną kartę rozszerzeń ze złączem HDMI. Karta ta łączy piny FPGA bezpośrednio z pinami gniazda HDMI na płycie, co wymusiło zaimplementowanie pełnej logiki kontrolera wideo bezpośrednio w strukturze układu programowalnego.

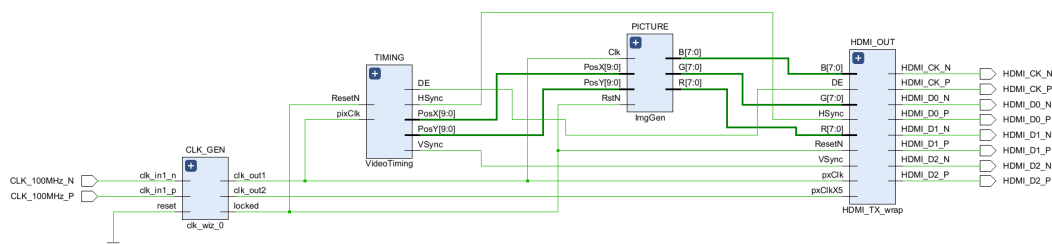
### 1.4 Architektura projektu i użyte moduły

Projekt został zrealizowany w sposób modułowy, rozdzielając logikę czasową od logiki generacji obrazu. Wyróżniono następujące bloki:

- **Clocking Wizard (CLK\_GEN):** Moduł wygenerowany za pomocą narzędzi IP Vivado. Wykorzystuje wbudowane w układ FPGA pętle fazowe (PLL/MMCM), aby z wejściowego zegara systemowego 100 MHz wygenerować dwa wysoce stabilne i zsynchronizowane zegary fazowe: `clk_25M` (25 MHz) taktujący logikę generowania pikseli oraz `clk_125M` (125 MHz) wykorzystywany przez układ serializujący TMDS.
- **VideoTiming (TIMING):** Dwuwymiarowy licznik współrzędnych ekranowych. Składa się z procesów sekwencyjnych generujących aktualne współrzędne `PosX` i `PosY` oraz ściśle sterujących sygnałami HSync, VSync i DE zgodnie z normami czasowymi standardu 640x480.
- **ImgGen (PICTURE):** Logika kombinacyjna i sekwencyjna pełniąca rolę generatora obrazu i sprzętowego multipleksera.
- **HDMI\_TX\_wrap (HDMI\_OUT):** Moduł IP nadajnika warstwy fizycznej (od prowadzącego). Dokonuje kodowania TMDS sygnałów RGB oraz sygnałów synchronizacji i sprzętowo serializuje je do złącza.

## 2 Schemat blokowy projektu (Block Design)

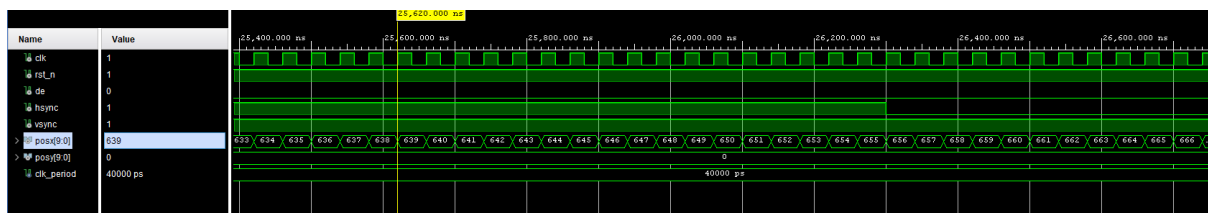
Poniższy schemat przedstawia architekturę typu Top-Level (`hdmi_top`) wygenerowaną za pomocą narzędzia RTL Analysis w środowisku Vivado. Ilustruje on przepływ sygnałów i zależności między wyżej opisanymi modułami sprzętowymi.



Rysunek 1: Schemat blokowy RTL modułu głównego hdmi\_top

### 3 Weryfikacja symulacyjna (Testbench)

Projektowanie układów FPGA wymaga dogłębnej weryfikacji logicznej przed synteza. Przeprowadzono symulację behawioralną modułu generującego czasy wideo (VideoTiming). Stworzono środowisko testowe (tb\_video.vhd), które symulowało sztuczny oscylator 25 MHz oraz wymuszało sygnał resetu.



Rysunek 2: Wykresy czasowe z symulacji układu VideoTiming

Jak widać na powiększeniu z symulacji (Rys. 2), struktury licznikowe posx i posy działają zgodnie z założeniami. Istotnym dowodem poprawności jest zachowanie sygnału DE (Display Enable). Gdy licznik pozycji poziomej kończy obszar aktywny (przekracza wartość 639), sygnał DE natychmiast przechodzi w stan niski, zapobiegając nadawaniu danych w strefie Front Porch.

### 4 Wdrożenie i weryfikacja sprzętowa

Projekt został poddany procesom syntezy i implementacji w środowisku Vivado. Zdefiniowano plik ograniczeń projektowych (hdmi\_pins.xdc), mapujący sygnały wejściowe zegara i różnicowe pary wyjściowe dla złącza HDMI w standardzie LVDS i LVCMOS33.



Rysunek 3: Wygenerowany sprzętowo domyślny obraz widoczny na monitorze HDMI

#### 4.1 Rozszerzenie projektu: Sprzętowy multiplekser obrazów testowych

W ramach rozbudowy projektu podstawowego, generator obrazu (ImgGen) został przekształcony w sprzętowy multiplekser wideo. Moduł otrzymał nowe, dwubitowe wejście sterujące (*Mode*), przypisane na poziomie pliku *constraints* do fizycznych przełączników suwakowych (Slide Switches) znajdujących się na płycie bazowej (piny AB1 i AF1). Zastosowano standard napięciowy LVCMOS12 dedykowany dla tych komponentów.

Za pomocą instrukcji warunkowej *case*, układ logiczny w locie (bez opóźnień) wybiera odpowiedni algorytm generowania pikseli w zależności od pozycji przełączników:

- **Tryb 00:** Domyślny, animowany kwadrat z tłami gradientowymi.
- **Tryb 01 (Color Bars):** Pionowe pasy testowe. Efekt osiągnięto poprzez bezpośrednie mapowanie starszych bitów licznika pikseli *PosX* na kanały RGB.
- **Tryb 10 (Checkerboard):** Szachownica o wymiarach 32x32 piksele. Do jej wygenerowania użyto zoptymalizowanej sprzętowo operacji logicznej XOR na 5. bicie liczników pozycji poziomej i pionowej, co całkowicie wyeliminowało konieczność użycia dzielników.
- **Tryb 11:** Ekran wygaszony (sygnały czerni).

Poniższe fotografie dokumentują poprawne działanie multipleksera podczas przełączania trybów pracy bezpośrednio na fizycznym sprzęcie:



Rysunek 4: Tryb 01: Pionowe pasy testowe (Color Bars)



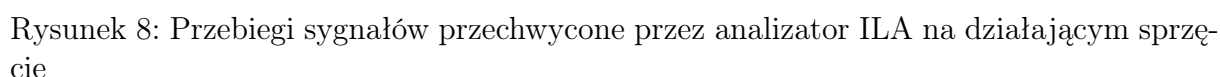
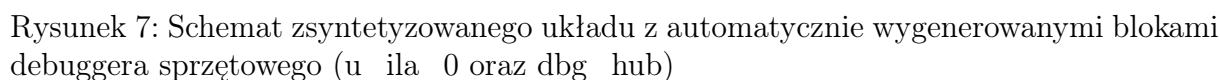
Rysunek 5: Tryb 10: Szachownica wygenerowana przy pomocy operacji XOR



Rysunek 6: Tryb 11: Bezpiecznik wygaszający ekran (czarny obraz)

## 5 Weryfikacja sprzętowa z użyciem analizatora ILA

W celu pogłębionej weryfikacji sprzętowej w środowisku wdrożeniowym, wykorzystano układ ILA (Integrated Logic Analyzer). Na wybrane sygnały magistrali wideo (m.in. HSync, VSync, PosX, PosY) nałożono atrybut *Mark for Debug*.



## 6 Podsumowanie i wnioski

Kluczowe wnioski płynące z realizacji projektu:

- 9

nych fazowo sygnałów zegarowych. Wykorzystanie wbudowanych bloków PLL (moduł Clocking Wizard) okazało się rozwiązaniem stabilnym i niezawodnym.

- **Przewaga generacji w locie:** Użycie logiki kombinacyjnej do wyliczania kolorów na bieżąco, na podstawie piksela wskazywanego przez licznik współrzędnych, pozwoliło pominąć konieczność stosowania obszernej pamięci VRAM jako bufora klatki, co oszczędza krytyczne zasoby układu FPGA.
- **Znaczenie standardów napięciowych (I/O Standards):** Podczas rozbudowy projektu o fizyczne przełączniki udowodniono, jak kluczowe jest ściśle mapowanie napięć. Przełączniki podłączone do części PL wymagały standardu 1.2V (LVCMOS12). Zadeklarowanie niewłaściwego standardu lub podpięcie nieużywanych pinów skutkuje błędną interpretacją stanów logicznych (tzw. floating inputs), co udało się z powodzeniem zdiagnozować i skorygować w pliku ograniczeń *constraints*.

Otrzymany obraz na interfejsie HDMI dowodzi, że integracja niestandardowych modułów RTL z gotowym rdzeniem warstwy fizycznej nadajnika została zrealizowana w 100% poprawnie.

## Literatura

- [1] VESA (Video Electronics Standards Association), *VESA Display Monitor Timing (DMT) Standard*.
- [2] HDMI Licensing, LLC, *High-Definition Multimedia Interface Specification Version 1.4b*.
- [3] Pong P. Chu, *FPGA Prototyping by VHDL Examples: Xilinx MicroBlaze Vitis Edition*, John Wiley & Sons, 2017.
- [4] AMD / Xilinx, *Clocking Wizard LogiCORE IP Product Guide (PG065)*.
- [5] AMD / Xilinx, *Zynq UltraScale+ Device Technical Reference Manual (UG1085)*.
- [6] dr. inż Jarosław Sugier, *Materiały z wykładu oraz udostępnione kody..*

## 7 Kody źródłowe

### 7.1 Moduł główny (hdmi\_top.vhd)

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity hdmi_top is
5      Port (
6          -- Główny zegar różnicowy z płyty (100 MHz)
7          CLK_100MHz_P : in STD_LOGIC;
8          CLK_100MHz_N : in STD_LOGIC;
9
10         -- PRZELACZNIKI (Wybor trybu wyswietlania)
11         SW : in STD_LOGIC_VECTOR(1 downto 0);
12
13         -- Piny zlacza HDMI na plycie rozszerzen
14         HDMI_CK_P : out STD_LOGIC;
15         HDMI_CK_N : out STD_LOGIC;
16         HDMI_D2_P : out STD_LOGIC;
17         HDMI_D2_N : out STD_LOGIC;
18         HDMI_D1_P : out STD_LOGIC;
19         HDMI_D1_N : out STD_LOGIC;
20         HDMI_D0_P : out STD_LOGIC;
21         HDMI_D0_N : out STD_LOGIC
22     );
23 end hdmi_top;
24
25 architecture Behavioral of hdmi_top is
26
27     component clk_wiz_0
28     port (
29         clk_in1_p : in std_logic;
30         clk_in1_n : in std_logic;
31         clk_out1  : out std_logic;
32         clk_out2  : out std_logic;
33         reset     : in std_logic;
34         locked    : out std_logic
35     );
36 end component;
37
38 component VideoTiming
39     Port ( pixClk : in  STD_LOGIC;
40           ResetN  : in  STD_LOGIC;
41           DE      : out STD_LOGIC;
42           HSync   : out STD_LOGIC;
43           VSync   : out STD_LOGIC;
44           PosX    : out STD_LOGIC_VECTOR(9 downto 0);
45           PosY    : out STD_LOGIC_VECTOR(9 downto 0) );
46 end component;
47
48 component ImgGen
49     Port ( Clk : in  STD_LOGIC;
50           RstN : in  STD_LOGIC;
51           Mode : in  STD_LOGIC_VECTOR(1 downto 0);
52           PosX : in  STD_LOGIC_VECTOR(9 downto 0);
53           PosY : in  STD_LOGIC_VECTOR(9 downto 0);
54           R    : out STD_LOGIC_VECTOR(7 downto 0);
55           G    : out STD_LOGIC_VECTOR(7 downto 0);
56           B    : out STD_LOGIC_VECTOR(7 downto 0) );
57 end component;
58
59 component HDMI_TX_wrap
60     Port (
61         pxClk      : in  STD_LOGIC;
62         pxClkX5    : in  STD_LOGIC;
63         ResetN     : in  STD_LOGIC;
64         DE          : in  STD_LOGIC;
65         HSync       : in  STD_LOGIC;
66         VSync       : in  STD_LOGIC;
67         R           : in  STD_LOGIC_VECTOR( 7 downto 0 );
68         G           : in  STD_LOGIC_VECTOR( 7 downto 0 );
```

```

69         B      : in  STD_LOGIC_VECTOR( 7 downto 0 );
70         HDMI_D0_P : out STD_LOGIC;
71         HDMI_D0_N : out STD_LOGIC;
72         HDMI_D1_P : out STD_LOGIC;
73         HDMI_D1_N : out STD_LOGIC;
74         HDMI_D2_P : out STD_LOGIC;
75         HDMI_D2_N : out STD_LOGIC;
76         HDMI_CK_P : out STD_LOGIC;
77         HDMI_CK_N : out STD_LOGIC
78     );
79 end component;
80
81 signal clk_25M, clk_125M, pll_locked : std_logic;
82 signal sync_h, sync_v, vde : std_logic;
83 signal pos_x, pos_y : std_logic_vector(9 downto 0);
84 signal red, green, blue : std_logic_vector(7 downto 0);
85
86 begin
87
88     CLK_GEN : clk_wiz_0 port map (
89         clk_in1_p => CLK_100MHz_P,
90         clk_in1_n => CLK_100MHz_N,
91         clk_out1  => clk_25M,
92         clk_out2  => clk_125M,
93         reset     => '0',
94         locked    => pll_locked
95     );
96
97     TIMING : VideoTiming port map (
98         pixClk => clk_25M,
99         ResetN => pll_locked,
100        DE      => vde,
101        HSync   => sync_h,
102        VSync   => sync_v,
103        PosX    => pos_x,
104        PosY    => pos_y
105    );
106
107    PICTURE : ImgGen port map (
108        Clk  => clk_25M,
109        RstN => pll_locked,
110        Mode => SW,           -- Podpiecie fizycznych przełączników do logiki
111        PosX => pos_x,
112        PosY => pos_y,
113        R    => red,
114        G    => green,
115        B    => blue
116    );
117
118    HDMI_OUT : HDMI_TX_wrap port map (
119        pxClk      => clk_25M,
120        pxClkX5    => clk_125M,
121        ResetN     => pll_locked,
122        DE         => vde,
123        HSync      => sync_h,
124        VSync      => sync_v,
125        R          => red,
126        G          => green,
127        B          => blue,
128        HDMI_D0_P  => HDMI_D0_P,
129        HDMI_D0_N  => HDMI_D0_N,
130        HDMI_D1_P  => HDMI_D1_P,
131        HDMI_D1_N  => HDMI_D1_N,
132        HDMI_D2_P  => HDMI_D2_P,
133        HDMI_D2_N  => HDMI_D2_N,
134        HDMI_CK_P  => HDMI_CK_P,
135        HDMI_CK_N  => HDMI_CK_N
136    );
137
138 end Behavioral;

```

## 7.2 Plik środowiska testowego (tb\_video.vhd)

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity tb_video is
5 end tb_video;
6
7 architecture behavior of tb_video is
8     component VideoTiming
9         Port ( pixClk : in  STD_LOGIC;
10              ResetN  : in  STD_LOGIC;
11              DE      : out STD_LOGIC;
12              HSync   : out STD_LOGIC;
13              VSync   : out STD_LOGIC;
14              PosX     : out STD_LOGIC_VECTOR (9 downto 0);
15              PosY     : out STD_LOGIC_VECTOR (9 downto 0)
16            );
17     end component;
18
19     signal clk      : std_logic := '0';
20     signal rst_n    : std_logic := '0';
21     signal de       : std_logic;
22     signal hsync    : std_logic;
23     signal vsync    : std_logic;
24     signal posx     : std_logic_vector(9 downto 0);
25     signal posy     : std_logic_vector(9 downto 0);
26
27     constant clk_period : time := 40 ns;
28 begin
29     UUT: VideoTiming port map (
30         pixClk => clk,
31         ResetN => rst_n,
32         DE     => de,
33         HSync  => hsync,
34         VSync  => vsync,
35         PosX   => posx,
36         PosY   => posy
37     );
38
39     clk_process : process
40     begin
41         clk <= '0';
42         wait for clk_period/2;
43         clk <= '1';
44         wait for clk_period/2;
45     end process;
46
47     stim_proc: process
48     begin
49         rst_n <= '0';
50         wait for 100 ns;
51         rst_n <= '1';
52         wait;
53     end process;
54 end behavior;
```

### 7.3 Plik obostrzeń projektowych (hdmi\_pins.xdc)

```
1 # --- PRZELACZNIKI SUWAKOWE (Slide Switches) ---
2 set_property PACKAGE_PIN AB1 [get_ports {SW[0]}]
3 set_property IOSTANDARD LVCMOS12 [get_ports {SW[0]}]
4
5 set_property PACKAGE_PIN AF1 [get_ports {SW[1]}]
6 set_property IOSTANDARD LVCMOS12 [get_ports {SW[1]}]
7
8 # ZEGAR 100MHz z plyty
9 set_property PACKAGE_PIN D7 [get_ports {CLK_100MHz_P}]
10 set_property PACKAGE_PIN D6 [get_ports {CLK_100MHz_N}]
11 set_property IOSTANDARD LVDS [get_ports CLK_100MHz_*]
12 create_clock -period 10.000 -name Clk_100MHz [get_ports CLK_100MHz_P]
13
14 # ZLACZE HDMI z plyty rozszerzen
15 set_property PACKAGE_PIN AF11 [get_ports {HDMI_D2_P}]
16 set_property PACKAGE_PIN AG11 [get_ports {HDMI_D2_N}]
17 set_property PACKAGE_PIN AH11 [get_ports {HDMI_D1_P}]
18 set_property PACKAGE_PIN AH12 [get_ports {HDMI_D1_N}]
19 set_property PACKAGE_PIN AF10 [get_ports {HDMI_D0_P}]
20 set_property PACKAGE_PIN AE10 [get_ports {HDMI_D0_N}]
21 set_property PACKAGE_PIN AD12 [get_ports {HDMI_CK_P}]
22 set_property PACKAGE_PIN AC12 [get_ports {HDMI_CK_N}]
23 set_property IOSTANDARD LVCMOS33 [get_ports HDMI_*]
24
25 set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
```

## 7.4 Moduł generatora czasów wideo (VideoTiming.vhd)

```
1  -----
2  -- Basic Video Timing Generator
3  -- J.Sugier, PWr Edu PL
4  -----
5  library IEEE;
6  use IEEE.STD_LOGIC_1164.ALL;
7  use IEEE.NUMERIC_STD.ALL;
8
9  entity VideoTiming is
10     Port ( pixClk : in  STD_LOGIC;
11           ResetN  : in  STD_LOGIC;
12           DE      : out STD_LOGIC;
13           HSync   : out STD_LOGIC;
14           VSync   : out STD_LOGIC;
15           PosX    : out STD_LOGIC_VECTOR (9 downto 0);
16           PosY    : out STD_LOGIC_VECTOR (9 downto 0)
17     );
18 end VideoTiming;
19
20 architecture Behavioral of VideoTiming is
21
22     constant H_VISIBLE_PIXELS : integer := 640;
23     constant H_FRONT_PORCH    : integer := 16;
24     constant H_SYNC_PULSE     : integer := 96;
25     constant H_BACK_PORCH     : integer := 48;
26     constant V_VISIBLE_LINES  : integer := 480;
27     constant V_FRONT_PORCH    : integer := 10;
28     constant V_SYNC_PULSE     : integer := 2;
29     constant V_BACK_PORCH     : integer := 29;
30     constant H_SYNC_VALUE     : std_logic := '0';
31     constant V_SYNC_VALUE     : std_logic := '0';
32
33     constant LAST_VISIBLE_PX   : integer := H_VISIBLE_PIXELS - 1;
34     constant LAST_FPORCH_PX   : integer := LAST_VISIBLE_PX + H_FRONT_PORCH;
35     constant LAST_HSYNC_PX    : integer := LAST_FPORCH_PX + H_SYNC_PULSE;
36     constant LAST_BPORCH_PX   : integer := LAST_HSYNC_PX + H_BACK_PORCH;
37     constant LAST_VISIBLE_LN  : integer := V_VISIBLE_LINES - 1;
38     constant LAST_FPORCH_LN   : integer := LAST_VISIBLE_LN + V_FRONT_PORCH;
39     constant LAST_VSYNC_LN    : integer := LAST_FPORCH_LN + V_SYNC_PULSE;
40     constant LAST_BPORCH_LN   : integer := LAST_VSYNC_LN + V_BACK_PORCH;
41
42     signal cntX : unsigned( 9 downto 0 ) := ( others => '0' );
43     signal cntY : unsigned( 9 downto 0 ) := ( others => '0' );
44     signal cntX_TC, cntY_TC : std_logic;
45     signal DE_X, DE_Y : std_logic := '1';
46
47 begin
48
49     cntX_TC <= '1' when cntX = LAST_BPORCH_PX else '0';
50     process( pixClk )
51     begin
52         if rising_edge( pixClk ) then
53             if ResetN = '0' or cntX_TC = '1' then
54                 cntX <= ( others => '0' );
55             else
56                 cntX <= cntX + 1;
57             end if;
58         end if;
59     end process;
60
61     cntY_TC <= '1' when cntX_TC = '1' and cntY = LAST_BPORCH_LN else '0';
62     process( pixClk )
63     begin
64         if rising_edge( pixClk ) then
65             if ResetN = '0' or cntY_TC = '1' then
66                 cntY <= ( others => '0' );
67             elsif cntX_TC = '1' then
68                 cntY <= cntY + 1;
69             end if;
70         end if;
71     end process;
```

```

72
73     process( pixClk )
74     begin
75         if rising_edge( pixClk ) then
76             if ResetN = '0' or cntX = LAST_HSYNC_PX then
77                 HSync <= not H_SYNC_VALUE;
78             elsif cntX = LAST_FPORCH_PX then
79                 HSync <= H_SYNC_VALUE;
80             end if;
81         end if;
82     end process;
83
84     process( pixClk )
85     begin
86         if rising_edge( pixClk ) then
87             if ResetN = '0' or ( cntY = LAST_VSYNC_LN and cntX_TC = '1' ) then
88                 VSync <= not V_SYNC_VALUE;
89             elsif cntY = LAST_FPORCH_LN and cntX_TC = '1' then
90                 VSync <= V_SYNC_VALUE;
91             end if;
92         end if;
93     end process;
94
95     process( pixClk )
96     begin
97         if rising_edge( pixClk ) then
98             if ResetN = '0' or cntX = LAST_VISIBLE_PX then
99                 DE_X <= '0';
100             elsif cntX_TC = '1' then
101                 DE_X <= '1';
102             end if;
103         end if;
104     end process;
105
106     process( pixClk )
107     begin
108         if rising_edge( pixClk ) then
109             if ResetN = '0' or ( cntY = LAST_VISIBLE_LN and cntX_TC = '1' ) then
110                 DE_Y <= '0';
111             elsif cntY_TC = '1' then
112                 DE_Y <= '1';
113             end if;
114         end if;
115     end process;
116
117     DE <= DE_X and DE_Y;
118     PosX <= std_logic_vector( cntX );
119     PosY <= std_logic_vector( cntY );
120
121 end Behavioral;

```

## 7.5 Moduł generatora obrazu (ImgGen.vhd)

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity ImgGen is
6      Port (
7          Clk      : in STD_LOGIC;
8          RstN     : in STD_LOGIC;
9          Mode     : in STD_LOGIC_VECTOR(1 downto 0); -- Wybor trybu wyświetlania
10         PosX     : in STD_LOGIC_VECTOR (9 downto 0);
11         PosY     : in STD_LOGIC_VECTOR (9 downto 0);
12         R        : out STD_LOGIC_VECTOR (7 downto 0);
13         G        : out STD_LOGIC_VECTOR (7 downto 0);
14         B        : out STD_LOGIC_VECTOR (7 downto 0)
15     );
16 end ImgGen;
17
18 architecture Behavioral of ImgGen is
19
20     constant MODE_WIDTH : integer := 640;
21     constant MODE_HEIGHT : integer := 480;
22     constant SQ_SIZE : integer := MODE_HEIGHT / 10;
23
24     signal Sq_posX : signed (10 downto 0) := (others => '0');
25     signal Sq_posY : signed (10 downto 0) := (others => '0');
26     signal Sq_dX : signed (1 downto 0) := to_signed( 1, 2 );
27     signal Sq_dY : signed (1 downto 0) := to_signed( 1, 2 );
28     signal next_posX, next_posY : signed (10 downto 0);
29     signal sPosX, sPosY : signed(10 downto 0);
30
31 begin
32     sPosX <= signed( '0' & posX );
33     sPosY <= signed( '0' & posY );
34
35     -- MULTIPLESER OBRAZOW (Zalezy od stanu Mode)
36     process( sPosX, sPosY, Sq_posX, Sq_posY, PosX, Mode )
37     begin
38         case Mode is
39             -- TRYB 00: Oryginalny kwadrat i gradienty
40             when "00" =>
41                 if sPosX >= Sq_posX and sPosX < Sq_posX + SQ_SIZE and
42                    sPosY >= Sq_posY and sPosY < Sq_posY + SQ_SIZE then
43                     R <= (others => '1'); G <= (others => '1'); B <= (others => '1');
44                 else
45                     if sPosY < MODE_HEIGHT / 3 then
46                         R <= PosX( 7 downto 0 ); G <= (others => '0'); B <= (others =>
47                             '0');
48                     elsif sPosY < MODE_HEIGHT * 2 / 3 then
49                         R <= (others => '0'); G <= not PosX( 7 downto 0 ); B <= (others
50                             => '0');
51                     else
52                         R <= (others => '0'); G <= (others => '0'); B <= PosX( 7 downto 0
53                             );
54                     end if;
55                 end if;
56             -- TRYB 01: Pionowe pasy kolorowe
57             when "01" =>
58                 R <= (others => PosX(8));
59                 G <= (others => PosX(7));
60                 B <= (others => PosX(6));
61             -- TRYB 10: Szachownica czarno-biala 32x32 (Operacja XOR)
62             when "10" =>
63                 if (PosX(5) xor PosY(5)) = '1' then
64                     R <= (others => '1'); G <= (others => '1'); B <= (others => '1');
65                 else
66                     R <= (others => '0'); G <= (others => '0'); B <= (others => '0');
67                 end if;
68             -- TRYB 11: Pusty ekran (czarny)
```

```

69         when others =>
70             R <= (others => '0'); G <= (others => '0'); B <= (others => '0');
71         end case;
72     end process;
73
74     -- Animacja kwadratu (aktualizacja pozycji)
75     next_posX <= Sq_posX + Sq_dX;
76     next_posY <= Sq_posY + Sq_dY;
77
78     process( Clk )
79     begin
80         if rising_edge( Clk ) then
81             if RstN = '0' then
82                 Sq_posX <= (others => '0');
83                 Sq_posY <= (others => '0');
84                 Sq_dX <= to_signed( 1, 2 );
85                 Sq_dY <= to_signed( 1, 2 );
86             elsif sPosX = MODE_WIDTH - 1 and sPosY = MODE_HEIGHT - 1 then
87                 if next_posX < 0 then
88                     Sq_posX <= ( others => '0' ); Sq_dX <= to_signed( 1, 2 );
89                 elsif next_posX > MODE_WIDTH - SQ_SIZE then
90                     Sq_posX <= to_signed( MODE_WIDTH - SQ_SIZE, 11 ); Sq_dX <= to_signed(
91                         -1, 2 );
92                 else
93                     Sq_posX <= next_posX;
94                 end if;
95
96                 if next_posY < 0 then
97                     Sq_posY <= ( others => '0' ); Sq_dY <= to_signed( 1, 2 );
98                 elsif next_posY > MODE_HEIGHT - SQ_SIZE then
99                     Sq_posY <= to_signed( MODE_HEIGHT - SQ_SIZE, 11 ); Sq_dY <= to_signed(
100                         -1, 2 );
101                 else
102                     Sq_posY <= next_posY;
103                 end if;
104             end if;
105         end if;
106     end process;
107 end Behavioral;

```